

Matricola: _____

Nome: _____

Cognome: _____

ESAME Programmazione Avanzata e Parallela

15 giugno 2026

L'esame consiste in due parti:

- La **prima parte** consiste in 10 domande a risposta multipla sugli argomenti del corso. Ogni domanda può ricevere un punteggio massimo di *due* punti. Affinché una risposta sia considerata valida la scelta *deve essere motivata* in modo breve (non saranno infatti parte della consegna eventuali fogli “di brutta” o aggiuntivi). Una risposta errata o non motivata riceverà *zero* punti. *Il punteggio minimo è 12 punti su 20.*
- La **seconda parte** consiste in due esercizi di programmazione. Ogni esercizio può avere un punteggio massimo di *cinque* punti. Potete commentare brevemente il codice per chiarire le vostre scelte implementative. *Il punteggio minimo è 6 punti su 10.*

IMPORTANTE: Per raggiungere la sufficienza è necessario raggiungere il punteggio minimo in entrambe le parti.

Parte 1

Domanda 1

Si supponga di avere il seguente makefile:

```
all : main
```

```
main : A.o B.o C.o  
      gcc $^ -o main
```

```
%.o : %.c  
      gcc -c $< -o $@
```

Se, dopo aver precedentemente invocato `make` viene modificato il makefile stesso in modo da aggiungere `-fopenmp` alle opzioni di `gcc` e cancellato il file `A.o`, quali dei seguenti comandi verranno eseguiti alla prossima invocazione di `make`?

☐ gcc -fopenmp -c A.c -o A.o
gcc -fopenmp A.o B.o C.o -o main

☐ gcc -c A.c -o A.o
gcc A.o B.o C.o -o main

☐ gcc -fopenmp -c A.c -o A.o
gcc -fopenmp -c B.c -o B.o
gcc -fopenmp -c C.c -o C.o
gcc -fopenmp A.o B.o C.o D.o -o main

☐ Non verrà eseguito nessun comando

Domanda 2

Si supponga di avere questo frammento di codice C:

```
for (int i = 0; i < n; i++) {  
    for (int j = i+1; j < n; j++) {  
        if (v[i] > v[j]) {  
            m[i]++;  
        }  
    }  
}
```

e si assuma di avere variabili v , m e n definite e del tipo corretto. La variabile v punta a un vettore *ordinato* di float di n elementi uniformemente distribuiti tra 0 e 1. La variabile m punta a un vettore di int inizialmente tutti 0.

Quale delle seguenti è la versione branchless del codice precedente? La versione branchless migliorerebbe le prestazioni? Si assuma che il compilatore non ottimizzi.

☐ $m[i] += (v[i] > v[j]);$
Non migliorerebbe le prestazioni

☐ $int p = (v[i] <= v[j]);$
 $if (!p) m[i]++;$
Non migliorerebbe le prestazioni

☐ $m[i] += (v[i] > v[j]);$
Migliorerebbe le prestazioni

☐ $int p = (v[i] <= v[j]);$
 $if (!p) m[i]++;$
Migliorerebbe le prestazioni

Domanda 3

Si considerino le seguenti strutture:

```
struct X {  
    int a;  
    double b;  
    int c;  
};
```

```
struct Y {  
    double b;  
    int a;  
    int c;  
};
```

Quale delle seguenti affermazioni è corretta?

- | | |
|--|---|
| ☐ hanno necessariamente la stessa dimensione | ☐ hanno necessariamente dimensioni distinte |
| ☐ il compilatore può riorganizzare l'ordine dei campi se viene passata l'opzione <code>-O3</code> | ☐ potrebbero avere dimensioni distinte per questioni di allineamento |
-
-
-

Domanda 4

Si consideri il prodotto elemento per elemento due vettori accelerato con istruzioni SIMD (e.g., AVX2 o Neon). Quale delle seguenti affermazioni è corretta?

- | | |
|---|--|
| ☐ Le istruzioni SIMD distribuiscono il calcolo del prodotto tra più core | ☐ Le istruzioni SIMD effettuano il prodotto su blocchi di valori contigui |
| ☐ Le istruzioni SIMD serializzano le operazioni ed effettuano le operazioni un bit alla volta su stream di 128, 256 o 512 bit. | ☐ Le istruzioni utilizzano una cache e l'esecuzione speculativa per ottenere il risultato senza doverlo calcolare |
-
-
-

Domanda 5

Quale delle seguenti affermazioni sull'I/O è corretta?

- | | |
|--|---|
| ☐ <code>getc</code> può ritornare il valore EOF che per convenzione è definito come il valore massimo di un tipo <code>unsigned char</code> | ☐ <code>mmap</code> deve avere sempre un puntatore valido (non-NULL) come primo argomento che indichi l'indirizzo di base dove effettuare il mapping |
| ☐ <code>ftruncate</code> permette di far crescere la dimensione del file e non solo di troncarlo a dimensione minori | ☐ <code>fseek</code> con argomento indicante lo spostamento tale a 0 non cambia mai la posizione all'interno del file indipendentemente da quali siano gli altri argomenti |
-
-
-

Domanda 6

Dato il seguente codice C facente uso di OpenMP:

```
float f(float * M1, float * M2, float * v, int n)
{
    float s = 0;
    #TODO
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            M1[i * n + j] = M2[i * n + j] * M1[i * n + j] + v[i];
            s += M1[i * n + j];
        }
    }
    return s;
}
```

Si supponga che $M1$ e $M2$ siano matrici $n \times n$, e v un vettore di n elementi. Al posto del TODO che codice può essere inserito affinché il programma funzioni correttamente?

- | | |
|--|--|
| ☐ <code>pragma omp parallel for collapse(2) reduction(+ : s)</code> | ☐ <code>pragma omp parallel collapse(2)</code> |
| ☐ <code>pragma omp parallel reduction(+ : s)</code> | ☐ <code>pragma omp parallel for collapse(2) single</code> |
-
-
-

Domanda 7

Dato il seguente codice Python:

```
class A:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return A(x,y)

    def __str__(self):
        return f"{{self.x}} {{self.y}}"

class B(A):

    def __mul__(self, other):
        x = self.x * other.x
        y = self.y * other.y
        return B(x,y)

class C(B):

    def __add__(self, other):
        x = self.x + other.y
        y = self.y + other.x
        return A(x,y)
```

```
p = B(2, 3)
q = C(1, 7)
```

```
result = q + p*p
```

Quale delle seguenti affermazioni è corretta riguardo all'operazione $q + (p * p)$?

- | | |
|--|--|
| ☐ L'operazione <code>p * p</code> invoca <code>B.__mul__</code> , restituendo un oggetto di tipo <code>A</code> | ☐ L'operazione <code>q + ...</code> invoca <code>C.__add__</code> , ma l'oggetto restituito è di tipo <code>A</code> |
| ☐ L'operazione <code>q + ...</code> invoca <code>A.__add__</code> perché la classe <code>C</code> eredita da <code>A</code> | ☐ Viene sollevata un'eccezione <code>AttributeError</code> perché la classe <code>C</code> non definisce il metodo <code>__add__</code> |
-
-
-

Domanda 8

Dato il seguente codice Python:

```
def f(g, h):  
    def k(x):  
        return g(h(x), x)  
    return k
```

```
f1 = lambda x,y: x(y)  
f2 = lambda x: (lambda y: y + x)  
f3 = f(f1, f2)
```

Quale delle seguenti affermazioni descrive correttamente la funzione `f3`?

- | | |
|---|---|
| ☐ <code>f3</code> è una funzione che, quando chiamata con un argomento <code>x</code> , restituisce una funzione di zero argomenti | ☐ <code>f3</code> è una funzione che, quando chiamata con un argomento <code>x</code> , calcola <code>x + x</code> |
| ☐ <code>f3</code> è un oggetto di tipo <code>lambda</code> che non può accettare argomenti numerici | ☐ <code>f3</code> è equivalente alla funzione <code>f2</code> |
-
-
-

Domanda 9

Dato il seguente codice Python:

```
def deco(g):  
    def f(k):  
        def h(*args, **kwargs):  
            return g(k(*args, **kwargs))  
        return h  
    return f
```

```
@deco(lambda x: sum(x))
```

```
def k(n):  
    x = []  
    for i in range(n):  
        x.append(i)  
    return x
```

Quale è l'effetto del decoratore sulla funzione `k`?

- | | |
|--|---|
| ☐ Trasforma <code>k(n)</code> in un generatore che restituisce la somma parziale ad ogni iterazione | ☐ Modifica <code>k(n)</code> in modo che il valore restituito sia la somma degli elementi della lista generata da <code>k</code> |
| ☐ Applica la funzione <code>sum</code> agli argomenti passati a <code>k</code> prima che la funzione venga eseguita | ☐ Sostituisce la funzione <code>k(n)</code> con la funzione <code>lambda x: sum(x)</code> |
-
-
-

Domanda 10

Dato il seguente codice Python:

```
def f(lst, k):  
    n = 1  
    while True:  
        for x in lst:  
            if x < k:  
                yield x*n  
        n = n + 1
```

```
h = f([7, 8, 3, 2, 5, 1, 6], 4)  
x = 0  
for i in range(10):  
    x = next(h)
```

Cosa avviene durante l'esecuzione del codice?

- | | |
|--|---|
| ☐ Il generatore esaurisce la lista originale e solleva un'eccezione <code>StopIteration</code> | ☐ Il generatore restituisce in modo ciclico i valori della lista inferiori a <code>k</code> moltiplicati per un <code>n</code> crescente |
| ☐ Il generatore restituisce ciclicamente gli stessi valori della lista originale senza variazioni | ☐ Il generatore restituisce invariati solo i valori della lista che sono strettamente minori di <code>k</code> |
-
-
-

Parte 2

Esercizio 1

Si completi la seguente funzione C che ha i seguenti argomenti:

- `v1`, `v2` e `v3` sono vettori di lunghezza `n`;
- `vout` è un puntatore a un altro vettore di lunghezza `n` che dovrà essere riempito dalla funzione;
- `vmean` e `vvar` sono puntatori a singoli valori float che dovranno essere modificati dalla funzione.

La funzione deve calcolare ogni `vout[i]` come `v1[i] * v2[i] + v3[i]`, il valore puntato da `vmean` dovrà contenere il valore medio di `vout` e il valore `vvar` la sua varianza. Il riempimento di `vout` e il calcolo di `vmean` e `vvar` deve essere correttamente parallelizzato con OpenMP. È possibile utilizzare fino a un massimo di due cicli `for`.

```
void f(float * v1, float * v2, float * v3, int n,  
      float * vout, float * vmean, float * vvar)
```

```
{
```

```
    float mean = 0;
```

```
    float var = 0;
```

```
*vmean = mean;  
*vvar = var;  
}
```

Esercizio 2

Si scriva il metodo `__getitem__` della classe `A` che, dato un parametro `n` intero non negativo ritorni una funzione che consiste nella composizione di `n` volte della funzione `f` (passata come argomento al costruttore) con sé stessa. Per esempio, se $n = 3$, verrà ritornata una funzione che calcola $f \circ f \circ f$, i.e., $f(f(f(x)))$. Si assuma che `f` richieda un singolo argomento e che per $n = 0$ la funzione da ritornare sia l'identità.

class A:

```
def __init__(self, f):  
    self.f = f
```

```
def __getitem__(self, n):
```
